

UNDERGRADUATE LECTURE

Introduction to Relational Databases and SQL

Concepts, structure, and queries — from first principles to working PostgreSQL.

≈ 2 hours · 11 parts · PostgreSQL

OVERVIEW

Today's Roadmap

CONCEPTS

- 1** What is a database?
- 2** The relational model
- 3** ER modeling
- 4** Relational algebra

SQL

- 5** SELECT, WHERE, ORDER BY
- 6** Joins
- 7** Aggregation
- 8** Subqueries
- 9** Defining tables

DESIGN & RELIABILITY

- 10** Normalization
- 11** Transactions & ACID

PART 1

What Is a Database?

Data Is Everywhere

Your university

Enrollment, grades, tuition, timetables — every registrar runs on a database.

Your bank

Accounts and transfers, where losing or duplicating a record is unacceptable.

Every app on your phone

Messages, posts, orders, playlists — stored and queried millions of times a second.

Science & government

Genomes, climate records, censuses — datasets far too large for one file.

The common problem: **lots of structured data, many users, and a need for correctness.**

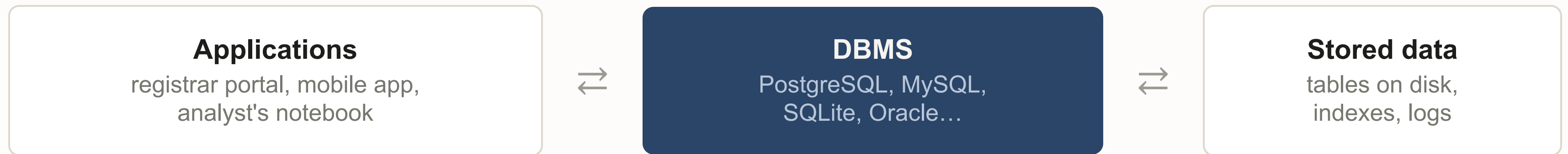
The File-Based Approach and Its Problems

Imagine the registrar keeps everything in spreadsheets: `students.xlsx`, `grades.xlsx`, one copy per department...

- ✗ **Redundancy & inconsistency** — the same student's address in five files, four of them stale.
- ✗ **Hard to query** — “which Math majors got an A in databases?” means writing a custom program.
- ✗ **No concurrent access** — two clerks editing the same file overwrite each other's work.
- ✗ **No integrity rules** — nothing stops a grade for a student who doesn't exist.
- ✗ **Fragile under failure** — a crash mid-update can corrupt the file silently.
- ✗ **No access control** — anyone with the file sees everything, including grades.

The Database Management System

A **database** is an organized collection of related data. A **DBMS** is the software that stores it, answers questions about it, and keeps it correct.



Applications never touch the files directly — they ask the DBMS, in **SQL**.

What a DBMS Provides

Declarative queries

Say *what* you want, not *how* to find it. The DBMS plans the search.

Data integrity

Declared rules — types, keys, constraints — are enforced on every change.

Concurrency

Thousands of simultaneous users, each seeing a consistent picture.

Durability & recovery

Committed data survives crashes; half-finished work is rolled back.

A Brief History of Databases

1960s

Hierarchical & network models — data navigated record by record, by hand-written code.

1970

Edgar F. Codd publishes the relational model: all data as tables, queried with mathematics.

1980s

SQL becomes the standard language; Oracle, DB2, and friends commercialize the model.

Today

Relational systems (PostgreSQL, MySQL, SQLite) remain the default home for structured data.

Fifty-five years later, Codd's idea is still the foundation. Let's see what it says.

PART 2

The Relational Model

Our Running Example: A University

Every example today uses the same tiny university. Three kinds of facts:

students

Who studies here — id, name,
major, year.

courses

What is taught — id, title, credits,
department.

enrollments

Who takes what — student,
course, semester, grade.

Small enough to fit on a slide — rich enough to show every idea in this lecture.

Tables, Rows, and Columns

students

SID	NAME	MAJOR	YEAR
1	Ada Lin	CS	2
2	Ben Osei	CS	3
3	Chloe Park	Math	1
4	Diego Ruiz	CS	4

A table holds facts of one kind — here, one row per student.

A row (highlighted) is one complete fact: “*Ben Osei is a 3rd-year CS student.*”

A column is one property, with the same meaning and type in every row.

A cell holds a single, indivisible value — no lists, no nested tables.

Formal Terms: Relation, Tuple, Attribute

EVERYDAY NAME	FORMAL NAME	IN OUR EXAMPLE
Table	Relation	students
Row / record	Tuple	(2, Ben Osei, CS, 3)
Column / field	Attribute	major
Number of rows	Cardinality	4
Number of columns	Degree (arity)	4

“Relational” database = database made of **relations**. Nothing more mysterious than tables.

Domains and Data Types

Every attribute has a **domain**: the set of values it may hold. In SQL, you declare the domain as a **data type** — and the DBMS rejects anything outside it.

sid

whole numbers
integer

name

text, up to 100 chars
varchar(100)

credits

small whole numbers
smallint

semester

dates or coded text
text • date

Atomic values only: a cell holds *one* number or *one* string — never a list. (This rule returns as “first normal form.”)

Schema vs. Instance

Schema — the blueprint

Table names, attribute names, types, keys. Designed once, changed rarely.

```
students(sid, name, major, year)
```

Instance — the contents today

The actual rows currently stored. Changes with every insert, update, delete.

```
(1, Ada Lin, CS, 2)  
(2, Ben Osei, CS, 3) ...
```

Like a form template vs. the filled-in forms — one design, many contents over time.

Keys: Identifying Rows Uniquely

Rows have no order and no hidden identity — a row **is** its values. So how do we point at *one particular student*?

- ✗ `name` — two students can be called Ada Lin.
- ✗ `name + major` — still no guarantee: two CS students named Ada Lin.
- ✓ `sid` — an ID the university assigns precisely so it is unique. This is a **key**.

A **key** is a minimal set of attributes whose values are guaranteed different in every row.

Primary Keys

A table may have several possible keys; we choose one as **the primary key** — written underlined. Every table should have one.

```
students(sid, name, major, year)
```

```
courses(cid, title, credits, dept)
```

```
enrollments(sid, cid, semester, grade)
```

Note enrollments: no single column is unique, so the key is **composite** — a student can retake a course, but not twice in the same semester.

Foreign Keys: Linking Tables

students

SID	NAME
1	Ada Lin
2	Ben Osei
3	Chloe Park

references

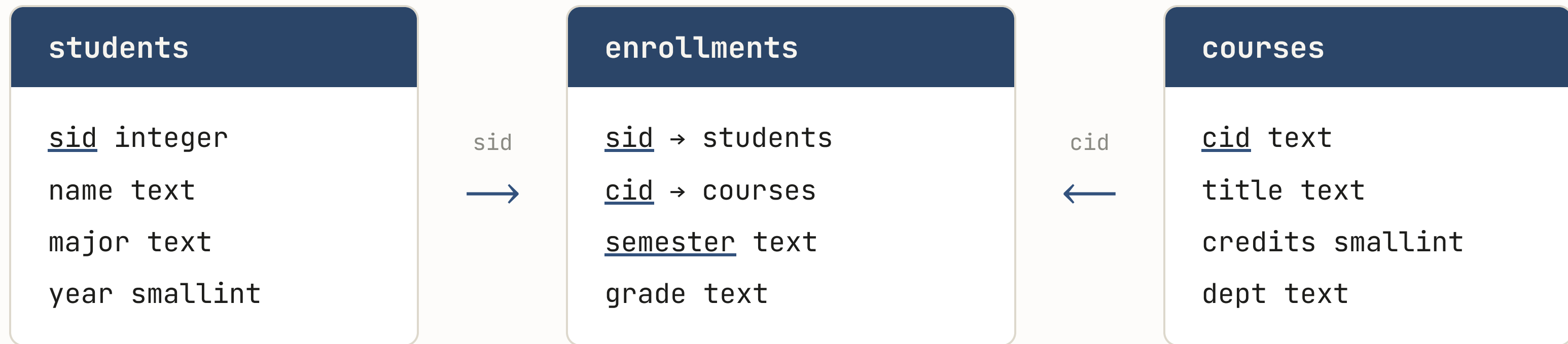


enrollments

SID	CID	GRADE
1	C101	A
2	C101	B+
99 ?	C102	A

A **foreign key** is a column whose values must appear as a key in another table. Student 99 doesn't exist — a DBMS enforcing **referential integrity** rejects that row.

The University Schema, Complete



Underline = primary key · arrow = foreign key reference. Keep this picture in mind all lecture.

Why Split Data into Tables?

Why not one big table? Watch what happens when Ada takes two courses:

NAME	MAJOR	YEAR	COURSE TITLE	CREDITS	GRADE
Ada Lin	CS	2	Intro to Databases	4	A
Ada Lin	CS	2	Linear Algebra	3	A-

Her details repeat in every row — change her major and you must update all of them, everywhere, consistently.

Facts get entangled — delete her last enrollment and Ada herself vanishes from the database.

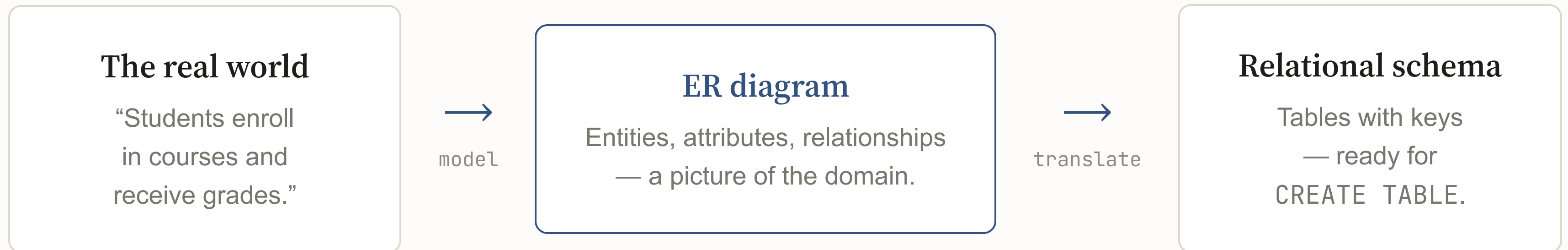
Splitting fixes it — each fact lives in exactly one place; keys link them back together on demand.

PART 3

Entity–Relationship Modeling

From the Real World to Tables

Where did our three tables come from? Good schemas aren't guessed — they are **designed**, in two steps:



The second step is nearly mechanical — the thinking happens in the diagram.

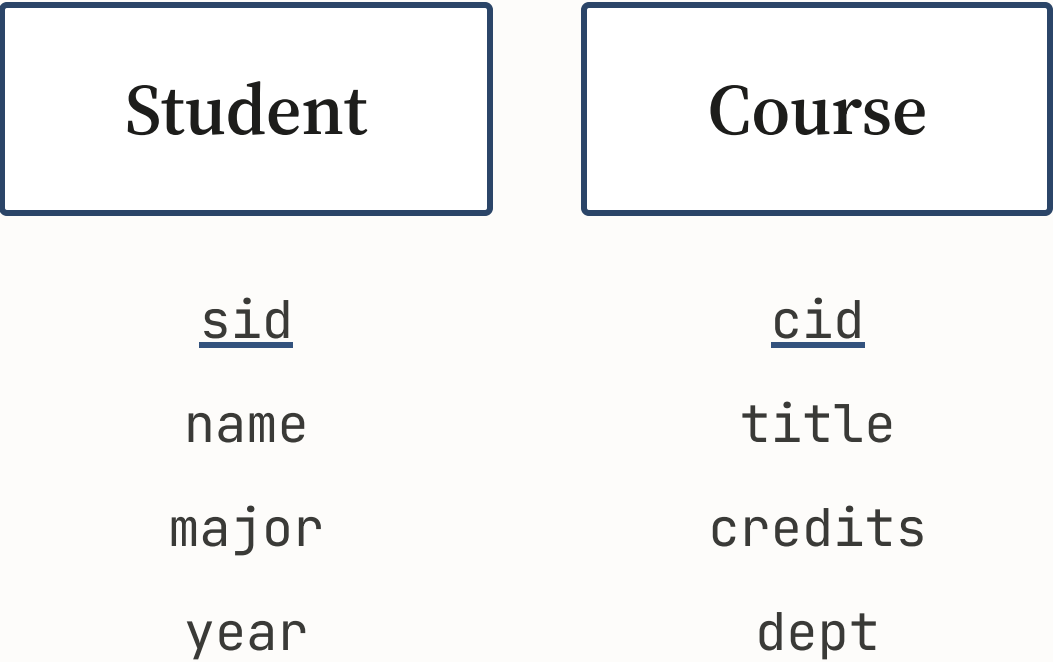
Entities and Attributes

An **entity** is a distinct thing we want to record facts about — *this* student, *that* course.

An **entity set** is all entities of one kind — drawn as a rectangle.

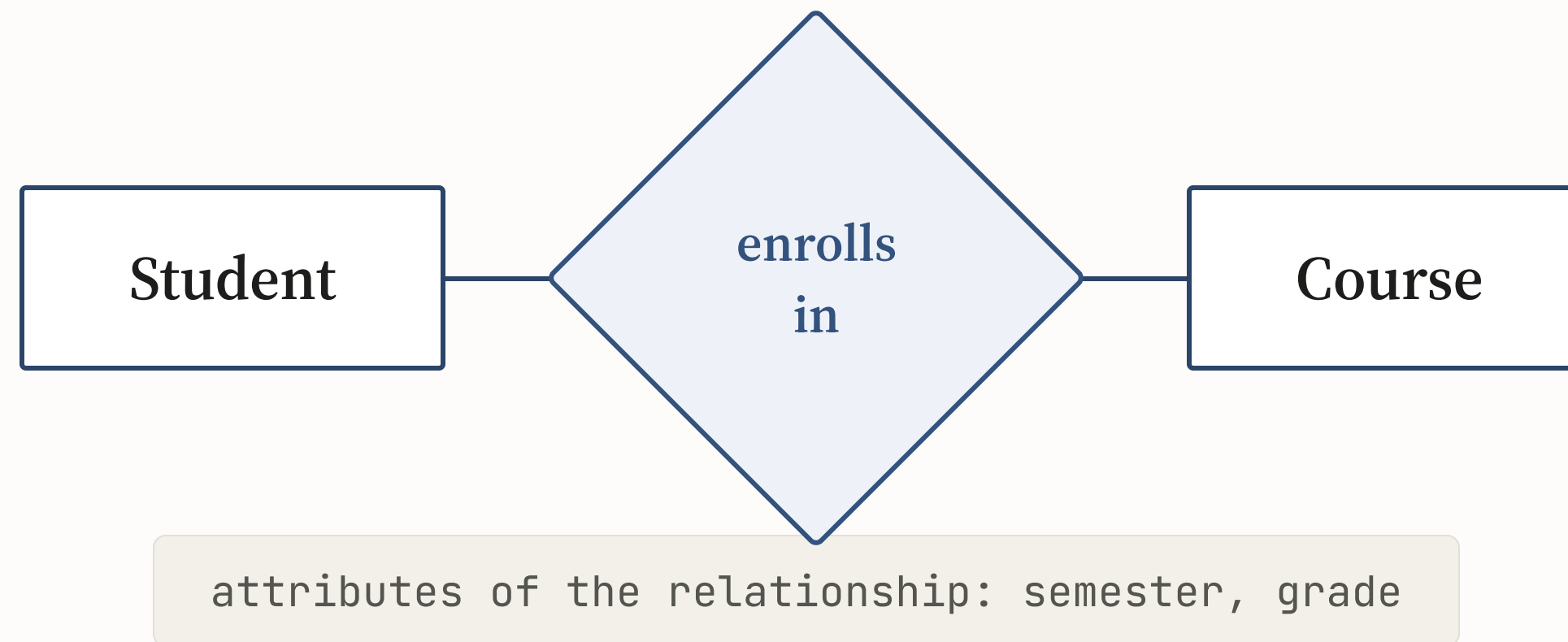
Attributes are the properties we record; the identifying one is underlined.

Rule of thumb: entities are the **nouns** in a description of the domain.



Relationships

A **relationship** is an association between entities — the **verbs** of the domain. Drawn as a diamond.



The grade belongs to the **pair** (student, course) — not to the student alone, nor the course alone. Relationships can carry attributes of their own.

Cardinality: How Many on Each Side?

1 : 1

One-to-one

Each student has one dorm-room contract; each contract names one student.

student — contract

1 : N

One-to-many

One department offers many courses; each course belongs to one department.

dept —< courses

M : N

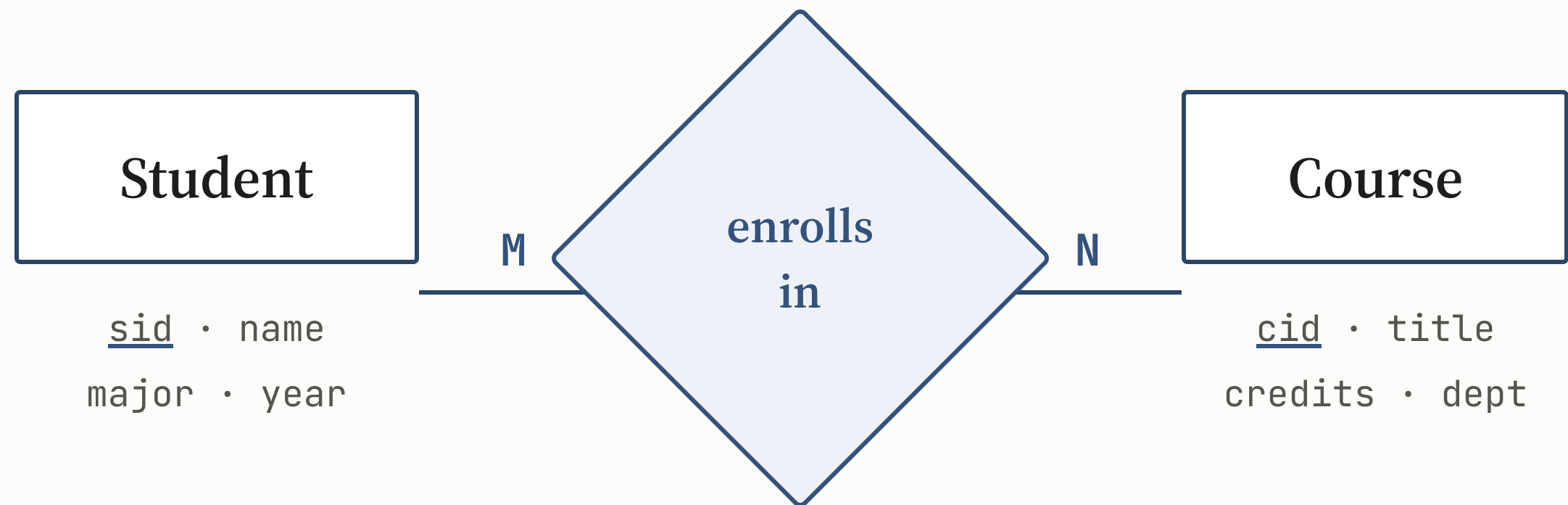
Many-to-many

A student takes many courses; a course has many students.
Our case!

students >—< courses

Getting cardinality right is the single most important design decision — it determines the table structure.

The University ER Diagram



semester · grade – attributes of "enrolls in"

Read it as a sentence: *"A student enrolls in a course, in some semester, receiving a grade."*

From ER Diagram to Tables

Rule 1 — each entity set becomes a table; its key becomes the primary key.



```
students(sid, name, major, year)  
courses(cid, title, credits, dept)
```

Rule 2 — each M:N relationship becomes its own table, keyed by *both* entity keys, carrying its attributes.



```
enrollments(sid, cid, semester, grade)
```

Rule 3 — a 1:N relationship needs no new table: put a foreign key on the “many” side.



```
courses(..., dept) – dept references departments
```

Our three-table schema is exactly what these rules produce. Design first, tables follow.

Many-to-Many Needs a Junction Table

✗ the tempting mistake

NAME	COURSES
Ada Lin	“C101, M201”

A list crammed into one cell: impossible to query (“who takes M201?”), impossible to attach a grade to each course, easy to corrupt.

✓ the relational way

SID	CID	GRADE
1	C101	A
1	M201	A-

One row per pairing. Each pairing can carry its own attributes. Everything stays queryable — this is the **junction table** pattern.

PART 4

Relational Algebra

Queries as Operations on Tables

Every operation takes one or two tables and **returns a table** — so operations compose, like arithmetic on numbers.

σ selection
keep some **rows**

π projection
keep some **columns**

$\bowtie$ join
combine two tables on
matching values

$\times$ product
all pairs of rows

$\cup \cap -$ set ops
union, intersection, difference

Why it matters
SQL is a friendly syntax over
exactly these operations

Selection (σ): Filtering Rows

$\sigma_{\text{major} = \text{'CS'}}(\text{students})$

NAME	MAJOR	YEAR
Ada Lin	CS	2
Ben Osei	CS	3
Chloe Park	Math	1
Diego Ruiz	CS	4



NAME	MAJOR	YEAR
Ada Lin	CS	2
Ben Osei	CS	3
Diego Ruiz	CS	4

Rows that satisfy the condition survive; the columns are untouched. In SQL, this becomes **WHERE**.

Projection (π): Choosing Columns

$\pi_{\text{name, major}}(\text{students})$

SID	NAME	MAJOR	YEAR
1	Ada Lin	CS	2
2	Ben Osei	CS	3
3	Chloe Park	Math	1

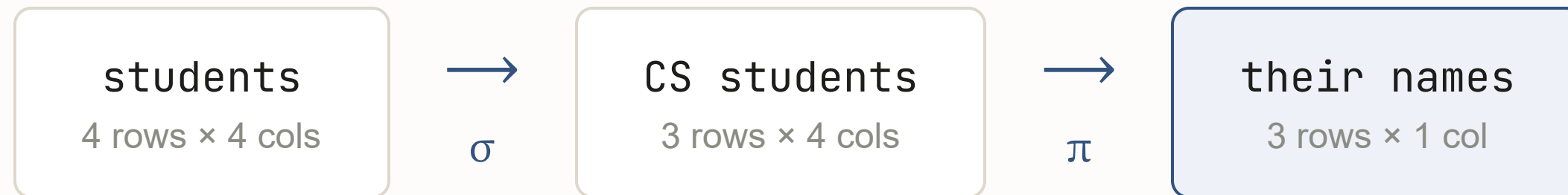
NAME	MAJOR
Ada Lin	CS
Ben Osei	CS
Chloe Park	Math

Columns are kept; rows pass through. In SQL, this is the **SELECT** list. σ picks rows; π picks columns — remember the pair.

Combining Selection and Projection

“The names of all CS students” — compose the two operations, reading inside-out:

$$\pi_{\text{name}}(\sigma_{\text{major} = \text{'CS'}}(\text{students}))$$



The same query in SQL: **SELECT** name **FROM** students **WHERE** major = 'CS'

Cartesian Product and Join

× Product

Every row of A paired with **every** row of B. 4 students
× 5 enrollments = 20 rows, most of them meaningless.

(Ada, ...) paired with Ben's enrollment ×

⋈ Join

Product **followed by selection**: keep only
pairs where the keys match. Only meaningful
combinations survive.

`students ⋈ enrollments on sid = sid ✓`

$$\text{students} \bowtie_{\text{sid}} \text{enrollments} = \sigma_{\text{match}}(\text{students} \times \text{enrollments})$$

The single most important operation for multi-table data — all of Part 6 is about it.

Set Operations: Union, Intersection, Difference

Let **A** = students in *Databases*, **B** = students in *Linear Algebra*. Same columns — so set operations apply:

$$A \cup B$$

Union

everyone in **either** course

$$A \cap B$$

Intersection

everyone taking **both**

$$A - B$$

Difference

in Databases but **not**
Linear Algebra

SQL spells these **UNION**, **INTERSECT**, **EXCEPT** — requirement: both inputs have compatible columns.

Why Relational Algebra Matters

A mental model for SQL

Every SQL query is a composition of these few operations. When a query confuses you, ask: *which rows am I keeping (σ)? which columns (π)? what am I joining ($\bowtie$)?*

The engine of optimization

Algebraic laws let the DBMS rewrite your query into a cheaper equivalent — filter early, join late. You declare *what*; the optimizer chooses *how*.

With the theory in hand — time to write actual queries.

PART 5

SQL Basics

SQL at a Glance

Declarative

Describe the result you want; the DBMS finds the fastest way to compute it.

Standardized

One core language across PostgreSQL, MySQL, SQLite, Oracle... We use **PostgreSQL**.

DDL — definition

CREATE, **ALTER**, **DROP** — build the schema (Part 9).

DML — manipulation

SELECT, **INSERT**, **UPDATE**, **DELETE** — use the data (now).

Conventions: keywords in UPPERCASE (by style, not necessity) · strings in 'single quotes' · statements end with ;

Anatomy of a SELECT Statement

```
SELECT name, year  
FROM students  
WHERE major = 'CS'  
ORDER BY year DESC  
LIMIT 10;
```

SELECT — which columns (π)

FROM — which table(s)

WHERE — which rows (σ)

ORDER BY — in what order

LIMIT — how many

Only **SELECT** and **FROM** are required. Mentally read **FROM** first: *take the table, filter the rows, keep the columns.*

SELECT: Choosing Columns

Everything — every column, every row:

```
SELECT *  
FROM students;
```

Just the columns you name, in that order:

```
SELECT name, major  
FROM students;
```

result of the second query

NAME	MAJOR
Ada Lin	CS
Ben Osei	CS
Chloe Park	Math
Diego Ruiz	CS

* is fine for exploring; in real code, name your columns — schemas change.

WHERE: Filtering Rows

```
SELECT name, year
FROM students
WHERE year ≥ 3;
```

The condition is checked **for each row**;
rows where it is true appear in the result,
the rest are silently dropped.

result – juniors and seniors only

NAME	YEAR
Ben Osei	3
Diego Ruiz	4

This is σ from the algebra, wearing its SQL clothes.

Comparison and Logical Operators

OPERATOR	MEANING
= <>	equal · not equal (one =, not ==)
< ≤ > ≥	comparisons, also on text and dates
BETWEEN a AND b	inclusive range
IN (...)	member of a list
AND OR NOT	combine conditions

Second- and third-year CS or Math students:

```
SELECT name
FROM students
WHERE year BETWEEN 2 AND 3
      AND major IN ('CS', 'Math');
```

Parenthesize when mixing **AND/OR** — a AND (b OR c) ≠ (a AND b) OR c.

Pattern Matching with LIKE

All courses with “Intro” in the title:

```
SELECT title
FROM   courses
WHERE  title LIKE '%Intro%';
```

PostgreSQL bonus: **ILIKE** matches case-insensitively —
'%intro%' then also finds “INTRO”.

PATTERN	MATCHES
'Intro%'	starts with “Intro”
'%bases'	ends with “bases”
'%data%'	contains “data”
'C1__'	“C1” + exactly two characters

% = any sequence (including empty) · _ = exactly one character

Working with NULL

Diego's course is still in progress — his grade is **NULL**: not zero, not an empty string, but “**no value here.**”

✗ finds nothing — ever

```
WHERE grade = NULL
```

Comparing anything with NULL gives *unknown*, never true — even `NULL = NULL`.

✓ the special syntax

```
WHERE grade IS NULL  
WHERE grade IS NOT NULL
```

NULL gets its own test. Rule of thumb: the moment NULLs exist in a column, think twice about every condition on it.

SQL uses **three-valued logic**: conditions are true, false, or *unknown* — and **WHERE** keeps only the **true** ones.

ORDER BY: Sorting Results

```
SELECT  name, major, year
FROM    students
ORDER BY year DESC, name ASC;
```

Sort by year, newest students last... then break ties alphabetically.

ASC is the default; **DESC** reverses.

result

NAME	MAJOR	YEAR
Diego Ruiz	CS	4
Ben Osei	CS	3
Ada Lin	CS	2
Chloe Park	Math	1

Without **ORDER BY**, row order is **undefined** — never rely on it.

LIMIT and DISTINCT

Top-N queries — the two most senior students:

```
SELECT  name, year
FROM    students
ORDER BY year DESC
LIMIT   2;
```

LIMIT without **ORDER BY** gives an *arbitrary* two rows — always pair them.

De-duplication — which majors exist?

```
SELECT DISTINCT major
FROM students;
```

Plain **SELECT major** returns CS three times. **DISTINCT** collapses duplicates → CS, Math.

Expressions and Aliases

Compute in the SELECT list; name results with **AS**:

```
SELECT title,  
       credits * 15 AS contact_hours  
FROM   courses;
```

result

TITLE	CONTACT_HOURS
Intro to Databases	60
Data Structures	60
Linear Algebra	45

Tables can be aliased too: **FROM** students **AS** s — then write s.name. Essential once several tables are in play.

Putting It Together

“The three most senior CS students, alphabetically within a year.” One clause at a time:

```
SELECT    name, year
FROM      students
WHERE     major = 'CS'
ORDER BY  year DESC, name
LIMIT     3;
```

- 1 • **FROM** take the students table
- 2 • **WHERE** keep CS majors
- 3 • **SELECT** keep name and year
- 4 • **ORDER BY** sort, seniors first
- 5 • **LIMIT** cut to three rows

Note: the DBMS **evaluates** in this order, not in the order you write. More on this in Part 7.

PART 6

Joins

Why We Need Joins

“*Which students got an A in Intro to Databases?*”

Student names live in `students` · grades live in `enrollments` · course titles live in `courses` .

We split the data on purpose — each fact once, in one place. The price: answering real questions means **recombining** tables.

A **join** stitches rows of two tables together wherever a condition holds — usually *foreign key = primary key*.

Normalize for storage, join for questions — that's the relational bargain.

The INNER JOIN

```
SELECT s.name, e.cid, e.grade
FROM   students AS s
JOIN   enrollments AS e
      ON s.sid = e.sid;
```

JOIN names the second table.

ON says when two rows belong together — here, same student id.

Each matching pair becomes one output row carrying **columns from both sides**.

Aliases (s, e) disambiguate: both tables have a column called `sid`.

JOIN means **INNER JOIN**: only rows with a match on **both** sides survive.

Join, Step by Step

students

SID	NAME
1	Ada Lin
2	Ben Osei
3	Chloe Park

enrollments

SID	CID	GRADE
1	C101	A
1	M201	A-
2	C101	B+



joined result

NAME	CID	GRADE
Ada Lin	C101	A
Ada Lin	M201	A-
Ben Osei	C101	B+

· Ada matches **two** enrollment rows → she appears **twice**.

· Chloe matches **none** → she disappears. (Remember her — she returns with LEFT JOIN.)

Joining Three Tables

Our opening question — “*who got an A in Intro to Databases?*” — needs the whole chain:

```
SELECT s.name
FROM   students      AS s
JOIN   enrollments AS e ON s.sid = e.sid
JOIN   courses       AS c ON e.cid = c.cid
WHERE  c.title = 'Intro to Databases'
      AND e.grade = 'A';
```



Each **JOIN** follows one foreign key. The joins rebuild the full picture; the **WHERE** narrows it to the rows we asked about.

Pattern to memorize: **entity** ⋈ **junction** ⋈ **entity**.

LEFT OUTER JOIN

The inner join lost Chloe — she isn't enrolled in anything. **LEFT JOIN** keeps **every** row of the left table:

```
SELECT s.name, e.cid, e.grade
FROM   students AS s
LEFT JOIN enrollments AS e
      ON   s.sid = e.sid;
```

NAME	CID	GRADE
Ada Lin	C101	A
Ada Lin	M201	A-
Ben Osei	C101	B+
Chloe Park	NULL	NULL

No match on the right → the right-hand columns are filled with **NULL**.

Classic trick — “students enrolled in **nothing**”: add **WHERE e.sid IS NULL** to the query above.

RIGHT and FULL OUTER JOIN

RIGHT JOIN

Mirror image of LEFT: every row of the **right** table survives; unmatched left side becomes NULL.

Rarely written in practice — swap the table order and use LEFT instead.

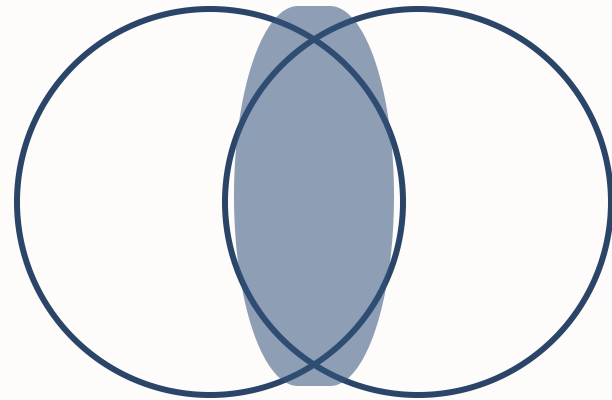
FULL JOIN

Keeps **every** row from **both** sides; NULLs fill whichever half has no partner.

Useful for reconciliation: “show all students and all enrollments, matched where possible.”

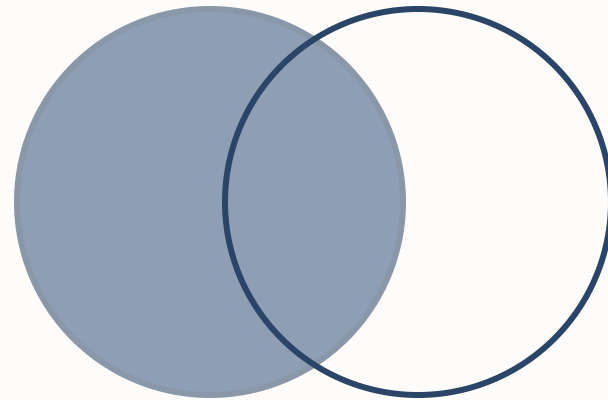
Everyday reality: **90% INNER, 9% LEFT, 1% everything else.**

A Visual Guide to Join Types



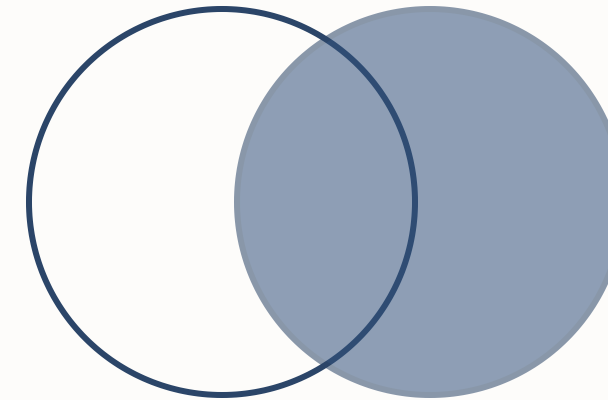
INNER

only matched rows



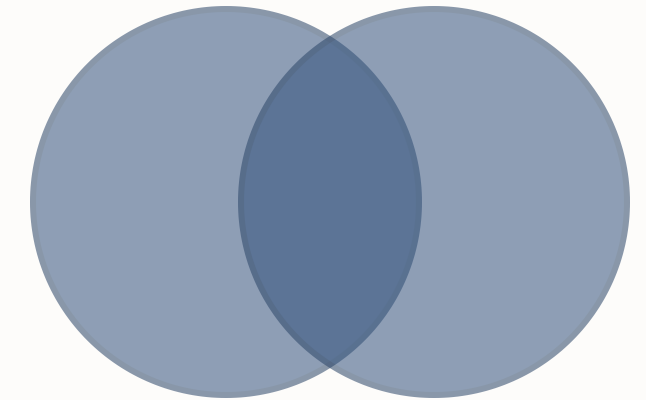
LEFT

all left rows + matches



RIGHT

all right rows + matches



FULL

everything, matched
where possible

Shaded = rows that appear in the result. Unmatched rows carry NULLs for the other side's columns.

Self Joins

Sometimes both sides of a join are the **same table** — “find pairs of students with the same major”:

```
SELECT a.name, b.name, a.major
FROM   students AS a
JOIN   students AS b
      ON a.major = b.major
AND    a.sid < b.sid;
```

Aliases a and b give us **two independent copies** of the table to pair up.

The condition `a.sid < b.sid` removes self-pairs (Ada, Ada) and mirror duplicates (Ben, Ada) vs (Ada, Ben).

Same mechanism handles hierarchies: an employees table joined to itself on `manager_id`.

Common Join Mistakes

- 1 The forgotten join condition.** `FROM students, enrollments` with no matching `WHERE` is a cartesian product — 10,000 students × 50,000 enrollments = 500 million rows of nonsense.
- 2 Joining on the wrong columns.** The query runs, returns rows, looks plausible — and is silently wrong. Always join foreign key to the key it references.
- 3 Losing rows without noticing.** Inner join drops unmatched rows silently — a report of “all students and their grades” that omits everyone with no grades yet. When every left row must appear, say **LEFT JOIN**.

Sanity check for any join: **is the row count roughly what you expected?**

PART 7

Aggregation

Aggregate Functions

FUNCTION	ANSWERS
COUNT(*)	how many rows?
SUM(credits)	total?
AVG(credits)	average?
MIN(year) MAX(year)	smallest / largest?

How many students, and the average year:

```
SELECT COUNT(*), AVG(year)
FROM students;
```

COUNT	AVG
4	2.5

Many rows in, **one row out**. Note: COUNT(grade) skips NULLs — Diego's ungraded course wouldn't count.

GROUP BY: One Row per Group

“How many students **per major**?” — the word *per* means **GROUP BY**:

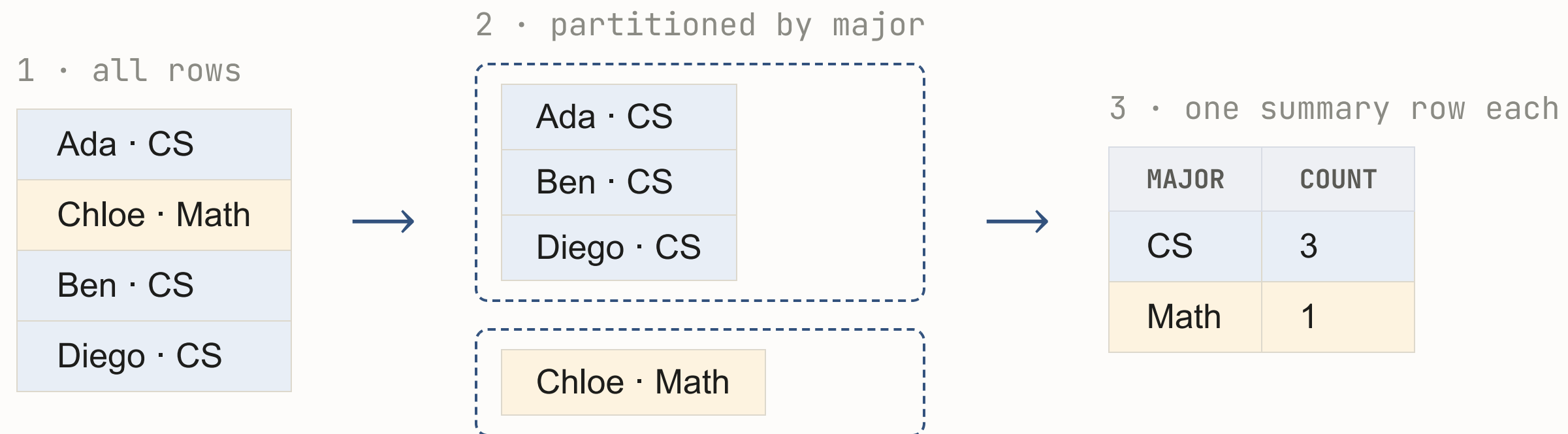
```
SELECT  major, COUNT(*)
FROM    students
GROUP BY major;
```

Rule: every SELECT column is either **the grouping column** or **inside an aggregate**. Bare name here would be an error — *which* name, out of three CS students?

result — one row per major

MAJOR	COUNT
CS	3
Math	1

GROUP BY, Step by Step



Partition, then collapse: the aggregate function runs **once per group**, not once per table.

HAVING: Filtering Groups

“Majors with at least two students”:

```
SELECT  major, COUNT(*)
FROM    students
GROUP BY major
HAVING  COUNT(*) ≥ 2;
```

result – Math's group is filtered out

MAJOR	COUNT
CS	3

HAVING runs **after** grouping, so it may use aggregates. **WHERE** runs before — it can't.

WHERE vs. HAVING

WHERE

Filters **rows**, **before** grouping.
Sees individual values; never aggregates.

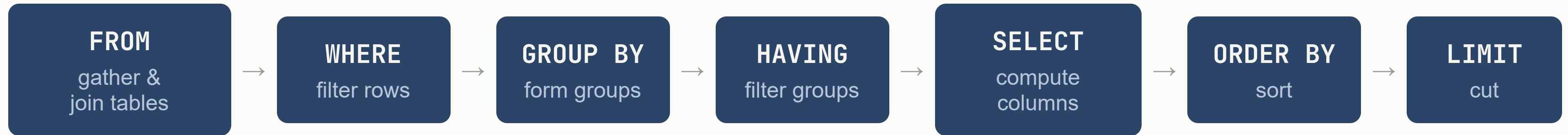
HAVING

Filters **groups**, **after** grouping.
Sees group summaries; aggregates welcome.

Both at once — upperclass students only, then well-populated majors only:

```
SELECT major, AVG(year) FROM students
WHERE year ≥ 2           -- rows first...
GROUP BY major
HAVING COUNT(*) ≥ 2;     -- ...groups after
```

The Full Query Pipeline



You **write** SELECT first, but it's **evaluated** sixth. This one picture explains most SQL puzzles:

- why **WHERE** can't use aggregates — groups don't exist yet
- why a column alias from **SELECT** can't be used in **WHERE** — it hasn't been computed yet

PART 8

Subqueries

A Query inside a Query

“Students above the average year.” The average isn't known in advance — **compute it in place**:

```
SELECT name, year
FROM students
WHERE year > (SELECT AVG(year)
              FROM students);
```

- 1 The inner query runs first → one value, 2.5.
- 2 The outer query behaves as if you'd written `year > 2.5`.

A **scalar subquery** — returns exactly one value.
Subqueries returning a *column* or a *table* come next.

Subqueries with IN

“Students enrolled in C101” — first find *the set of their ids*, then test membership:

```
SELECT name
FROM students
WHERE sid IN (SELECT sid
              FROM enrollments
              WHERE cid = 'C101');
```

Inner query → the set {1, 2} .

Outer query keeps students whose sid is in that set:
Ada, Ben.

The same question as a join — both are correct.

Rule of thumb: **need columns from both tables → join; only testing membership → IN reads cleaner.**

Correlated Subqueries

“The most senior student **in each major**.” The inner query now **refers to the outer row**:

```
SELECT name, major, year
FROM   students AS s
WHERE  year = (SELECT MAX(year)
              FROM students
              WHERE major = s.major );
```

The highlighted `s.major` comes from the **outer** row — the subquery is re-evaluated **for each student**.

For Diego (CS): inner query → max CS year = 4 → 4 = 4 ✓ he stays.

Read correlated subqueries as: *“for each row, ask a question about it.”*

Subqueries with EXISTS

EXISTS asks one thing: *does the subquery return any row at all?*

Students with at least one enrollment:

```
SELECT name FROM students s
WHERE EXISTS (SELECT 1
              FROM enrollments e
              WHERE e.sid = s.sid);
```

Students with none — Chloe again:

```
SELECT name FROM students s
WHERE NOT EXISTS (SELECT 1
                  FROM enrollments e
                  WHERE e.sid = s.sid);
```

Third way to ask “enrolled in nothing” — NOT EXISTS, NOT IN, or LEFT JOIN ... IS NULL. SQL usually offers several correct phrasings; pick the one that reads like the question.

Subqueries in FROM

A subquery result is a table — so it can sit in **FROM**. “Average number of courses per student”:

```
SELECT AVG(n)
FROM (SELECT sid, COUNT(*) AS n
      FROM enrollments
      GROUP BY sid) AS per_student;
```

Stage 1 inner query: one row per student with their course count.

Stage 2 outer query: average those counts.

Aggregating an aggregate needs two stages — one query can't do both at once. (Later you'll meet **WITH**, a more readable spelling of the same idea.)

PART 9

Defining Tables

CREATE TABLE

```
CREATE TABLE students (  
    sid      integer      PRIMARY KEY,  
    name     varchar(100) NOT NULL,  
    major    text,  
    year     smallint CHECK (year BETWEEN 1 AND 6)  
);
```

Each line: **column name**, **type**, then optional **constraints**.

The schema from Part 2, made executable — the design *is* the code.

Run it once; the empty table exists until dropped. Then INSERT fills it.

PostgreSQL Data Types — a Starter Set

TYPE	HOLDS	EXAMPLE
<code>integer · bigint</code>	whole numbers	42
<code>numeric(p,s)</code>	exact decimals — money, GPAs	3.85
<code>text · varchar(n)</code>	strings (varchar caps the length)	'Ada Lin'
<code>boolean</code>	true / false	true
<code>date · timestamptz</code>	calendar dates · moments in time	'2026-09-01'
<code>integer GENERATED ... AS IDENTITY</code>	auto-numbered ids for primary keys	1, 2, 3, ...

Choosing the right type is a free integrity check: a date column simply cannot hold “banana”.

Constraints: Rules the Database Enforces

NOT NULL

a value is required — no student without a name

UNIQUE

no duplicates — one account per email address

PRIMARY KEY

NOT NULL + UNIQUE — the row's official identity

CHECK (...)

any condition per row — year BETWEEN 1 AND 6

REFERENCES (foreign key)

values must exist in the referenced table — no enrollment for a phantom student

Declared once, enforced on **every insert and update, forever** — cheaper than cleaning bad data later.

Foreign Key Constraints in Action

```
CREATE TABLE enrollments (  
  sid      integer REFERENCES students(sid),  
  cid      text    REFERENCES courses(cid),  
  semester text,  
  grade    text,  
  PRIMARY KEY (sid, cid, semester)  
);
```

Inserting an enrollment for student 99:

```
ERROR: insert or update on table  
"enrollments" violates foreign key  
constraint "enrollments_sid_fkey"
```

And deleting a student *with* enrollments? You choose:
ON DELETE RESTRICT (refuse — the default) or **ON
DELETE CASCADE** (delete their enrollments too).

Changing and Removing Tables

```
ALTER TABLE students  
  ADD COLUMN email text UNIQUE;
```

Schemas evolve — add, rename, or re-type columns in place.

```
TRUNCATE enrollments;
```

Delete all rows, keep the table and its constraints.

```
DROP TABLE students;
```

Table, data, constraints — gone. **No undo.** Handle with care.

INSERT, UPDATE, DELETE

INSERT

```
INSERT INTO students  
  (sid, name, major, year)  
VALUES  
  (5, 'Emi Sato', 'CS', 1);
```

UPDATE

```
UPDATE students  
SET    major = 'Math'  
WHERE  sid = 5;
```

DELETE

```
DELETE FROM students  
WHERE sid = 5;
```

Forget the WHERE and you update or delete every row in the table. Habit worth forming: run the WHERE as a SELECT first, check the rows, then write.

PART 10

Normalization

Redundancy and Anomalies

Recall the one-big-table design, with Ada's details copied into every row. Redundancy breeds three named failures:

Update anomaly

Ada changes major — miss one of her rows and the database now contradicts itself.

Insert anomaly

A brand-new course with no students yet has no row to live in — no way to record it.

Delete anomaly

Drop Ada's last enrollment and Ada herself vanishes — deleting one fact destroyed another.

Normalization is the systematic cure: split tables until every fact is stored exactly once.

First Normal Form: Atomic Values

✗ violates 1NF

NAME	COURSES
Ada Lin	"C101, M201"

✓ in 1NF – one value per cell, one row per fact

NAME	CID
Ada Lin	C101
Ada Lin	M201

1NF: every cell holds a single, indivisible value — no lists, no repeating groups. The rule we've silently obeyed since Part 2.

Second Normal Form: No Partial Dependencies

Suppose we stored the course **title** inside `enrollments(sid, cid, semester, grade, title)` :

SID	CID	GRADE	TITLE
1	C101	A	Intro to Databases
2	C101	B+	Intro to Databases

The title depends on **cid** alone — only **part** of the key (sid, cid, semester). So it repeats for every enrolled student: redundancy again.

The fix: title lives in `courses` , keyed by cid — exactly our schema.

2NF: every non-key attribute depends on the **whole** key, not a part of it.

Third Normal Form: No Transitive Dependencies

Suppose `students` also stored each major's building:



The building doesn't describe the **student** — it describes the **major**. It reaches the key only *transitively*, through another non-key column.

Every CS row repeats “Gates Hall”; if CS moves, hundreds of rows must change. **Fix:** a `departments(major, building)` table.

Every attribute depends on **the key, the whole key, and nothing but the key** — the classic 3NF slogan.

Normalization in Practice

The default: normalize to 3NF

Each fact once. Updates touch one row. Constraints stay simple. For almost every application — course registries included — this is the right design.

The exception: deliberate denormalization

Read-heavy systems sometimes copy data to avoid joins — a **measured** trade of redundancy for speed, with the update burden accepted knowingly.

Order of operations: **design clean first, denormalize only when measurement demands it.**

PART 11

Transactions

The Need for Transactions

A student swaps *Linear Algebra* for *Data Structures* — two statements that must act as **one**:

```
DELETE FROM enrollments  
WHERE sid=1 AND cid='M201';
```

then

```
INSERT INTO enrollments  
VALUES (1, 'C102', 'F26', NULL);
```

Crash between them? The student has silently lost a course — the database is consistent-looking but wrong.

Someone reads between them? They see a student enrolled in nothing — a state that never “really” existed.

A **transaction** bundles statements into one all-or-nothing unit.

The ACID Properties

A Atomicity

All statements take effect, or none do — no half-finished swaps.

C Consistency

Every constraint holds before and after — never in between where you can see it.

I Isolation

Concurrent transactions can't see each other's unfinished work.

D Durability

Once committed, it survives crashes, power cuts, restarts.

These four guarantees are *why* banks, registrars, and airlines trust a DBMS instead of files.

Transactions in SQL

```
BEGIN;  
  
  DELETE FROM enrollments  
  WHERE sid=1 AND cid='M201';  
  
  INSERT INTO enrollments  
  VALUES (1, 'C102', 'F26', NULL);  
COMMIT;
```

BEGIN opens the transaction; changes are provisional.

COMMIT makes them permanent — atomically, all at once.

ROLLBACK instead abandons everything since **BEGIN** — your undo button.

Outside an explicit **BEGIN**, each statement is its own tiny auto-committed transaction.

What We Covered

Databases — shared, structured data with correctness guarantees a file can't give.

ER modeling — design the domain first; tables follow mechanically.

SQL basics — SELECT · FROM · WHERE · ORDER BY, and the care NULL demands.

Aggregation — GROUP BY partitions, aggregates collapse, HAVING filters groups.

DDL — CREATE TABLE with constraints: integrity declared, not hoped for.

Transactions — ACID: all-or-nothing, even under crashes and concurrency.

Relational model — everything is tables; keys identify, foreign keys connect.

Relational algebra — σ , π , $\bowtie$: the small calculus underneath every query.

Joins — recombining what normalization split; inner drops, outer keeps.

Subqueries — queries as values, sets, and tables inside other queries.

Normalization — the key, the whole key, and nothing but the key.

CLOSING

Where to Go from Here

Get your hands dirty — install PostgreSQL (or start with SQLite, zero setup) and rebuild today's university schema yourself.

Read further — any standard text: Silberschatz *Database System Concepts*, or the free PostgreSQL tutorial in the official docs.

Coming up in this course — indexing & query performance, concurrency control, and how the engine actually executes your SQL.

Thank you — questions welcome.